

Game Engine desde Cero – Informe de Progreso

Documento técnico que resume todo lo que has hecho hasta ahora: estructura de carpetas, código fuente, scripts de build, errores reales encontrados, soluciones y resultado final con una ventana SDL2 + OpenGL funcionando.

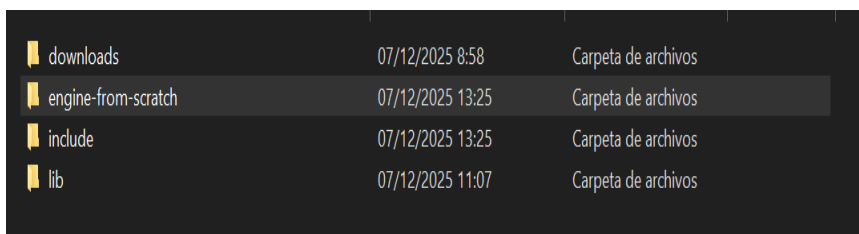
1. Estructura final del proyecto

La estructura de carpetas que has montado (y que funciona) es la siguiente. Es prácticamente la misma que la del vídeo, pero adaptada a tu ruta en E:\

E:\NACHO\ESTUDIO\PROYECTOS\PROGRAMACION\GAMEENGINE_GAME\

```
■
■ ■ ■ ■ engine-from-scratch\
■   ■ ■ ■ ■ build.bat
■   ■ ■ ■ ■ clean.bat
■   ■ ■ ■ ■ src\
■   ■   ■ ■ ■ ■ main.c
■   ■   ■ ■ ■ ■ glad.c
■   ■ ■ ■ ■ freetype.dll
■   ■ ■ ■ ■ SDL2.dll
■   ■ ■ ■ ■ SDL2_mixer.dll
■
■ ■ ■ ■ include\
■   ■ ■ ■ ■ glad\
■   ■   ■ ■ ■ ■ glad.h
■   ■   ■ ■ ■ ■ khrplatform.h
■   ■ ■ ■ ■ SDL2\
■   ■ ■ ■ ■ freetype\
■
■ ■ ■ ■ lib\
■   ■ ■ ■ ■ SDL2.lib
■   ■ ■ ■ ■ SDL2main.lib
■   ■ ■ ■ ■ freetype.lib
```

• ****engine-from-scratch/**** contiene el código del motor y los ejecutables. • ****include/**** agrupa todos los headers de las librerías externas. • ****lib/**** contiene las librerías estáticas contra las que enlaza el linker. • Las DLL viven junto al ejecutable para que Windows pueda cargarlas en tiempo de ejecución.



downloads	07/12/2025 8:58	Carpeta de archivos
engine-from-scratch	07/12/2025 13:25	Carpeta de archivos
include	07/12/2025 13:25	Carpeta de archivos
lib	07/12/2025 11:07	Carpeta de archivos

Captura: estructura de carpetas en la raíz de GAMEENGINE_GAME (downloads, engine-from-scratch, include, lib).

2. Código completo de main.c

```
#include <stdio.h>
#include <stdbool.h>
#include <glad/glad.h>
#define SDL_MAIN_HANDLED
#include <SDL2/SDL.h>

int main(int argc, char *argv[]) {

    SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, SDL_GL_CONTEXT_PROFILE_CORE);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, 3);
    SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, 3);

    if (SDL_Init(SDL_INIT_VIDEO) < 0) {
        printf("Could not init SDL: %s\n", SDL_GetError());
        exit(1);
    }

    SDL_Window *window = SDL_CreateWindow(
        "MyGame",
        SDL_WINDOWPOS_CENTERED,
        SDL_WINDOWPOS_CENTERED,
        800,
        600,
        SDL_WINDOW_OPENGL
    );

    if (!window) {
        printf("Failed to init window: %s\n", SDL_GetError());
        exit(1);
    }

    SDL_GL_CreateContext(window);

    if (!gladLoadGLLoader((GLADloadproc)SDL_GL_GetProcAddress)) {
        printf("Failed to load GL: %s\n", SDL_GetError());
        exit(1);
    }

    puts("OpenGL Loaded");
    printf("Vendor:      %s\n", glGetString(GL_VENDOR));
    printf("Renderer:    %s\n", glGetString(GL_RENDERER));
    printf("Version:      %s\n", glGetString(GL_VERSION));

    puts("Hello there!");

    bool should_quit = false;

    while (!should_quit){

        SDL_Event event;

        while (SDL_PollEvent(&event)) {
            switch (event.type) {
                case SDL_QUIT:
                    should_quit = true;
                    break;
                default:
                    break;
            }
        }

    }

    return 0;
}
```

2.1 Explicación línea a línea (por bloques)

****Líneas 1–4**** – *Includes y defines del entorno*:

- `#include`: funciones estándar de C como `printf`.
- `#include`: permite usar `bool`, `true` y `false` en C.
- `#include`: header principal de GLAD, que declara todas las funciones modernas de OpenGL.
- `#define SDL_MAIN_HANDLED`: le dice a SDL que no redefina `main` y que usaremos nuestro `main` normal.
- `#include`: header principal de SDL (ventanas, eventos, input, etc.).

****Línea 6**** – Firma de la función principal:

`int main(int argc, char *argv[])` es el punto de entrada del programa. `argc` y `argv` son los argumentos de línea de comandos.

****Líneas 8–10**** – Configuración del contexto OpenGL:

- `SDL_GL_SetAttribute(SDL_GL_CONTEXT_PROFILE_MASK, SDL_GL_CONTEXT_PROFILE_CORE);` pide un contexto Core (moderno, sin funciones deprecated).
- `SDL_GL_SetAttribute(SDL_GL_CONTEXT_MAJOR_VERSION, 3);` pide OpenGL 3.x.
- `SDL_GL_SetAttribute(SDL_GL_CONTEXT_MINOR_VERSION, 3);` pide específicamente 3.3.

****Líneas 12–16**** – Inicialización de SDL:

- `SDL_Init(SDL_INIT_VIDEO)` inicializa el subsistema de vídeo.
- Si devuelve un valor negativo, imprimimos el error (`SDL_GetError()`) y salimos con `exit(1)`.

****Líneas 18–26**** – Creación de la ventana:

- `SDL_CreateWindow` crea una ventana 800x600 titulada "MyGame".
- `SDL_WINDOWPOS_CENTERED` centra la ventana en la pantalla.
- `SDL_WINDOW_OPENGL` indica que la ventana soportará un contexto OpenGL.

****Líneas 28–31**** – Comprobación de la ventana:

Si `window` es NULL, algo falló al crear la ventana; imprimimos el error y salimos.

****Línea 33**** – Creación del contexto OpenGL:

`SDL_GL_CreateContext(window);` crea un contexto OpenGL y lo asocia a la ventana.

****Líneas 35–39**** – Carga de funciones OpenGL con GLAD:

- `gladLoadGLLoader((GLADloadproc)SDL_GL_GetProcAddress)` usa SDL para obtener las direcciones de las funciones de OpenGL.
- Si devuelve 0, significa que no se pudo cargar OpenGL y abortamos.

****Líneas 41–46**** – Mensajes de diagnóstico:

- `puts("OpenGL Loaded")` imprime un mensaje sencillo.
- `glGetString(GL_VENDOR)` devuelve el fabricante de la GPU (NVIDIA, AMD, etc.).
- `glGetString(GL_RENDERER)` el modelo concreto de GPU.
- `glGetString(GL_VERSION)` la versión de OpenGL que se está usando.
- `puts("Hello there!")` es simplemente una prueba de salida por consola.

****Líneas 48–50**** – Variable de control del bucle principal:

`bool should_quit = false;` se usa para controlar cuándo terminar el loop del juego.

****Líneas 52–65**** – Bucle principal:

- `while (!should_quit)` mantiene el programa corriendo mientras no se haya pedido salir.
- Dentro, declaramos un `SDL_Event event` para recibir eventos.
- `SDL_PollEvent(&event);` procesa todos los eventos pendientes.
- El `switch (event.type)` comprueba el tipo de evento.
- Si `event.type == SDL_QUIT`, significa que el usuario ha cerrado la ventana → ponemos `should_quit = true`.
- `default:` ignora el resto de eventos por ahora.

****Línea 67**** – Fin del programa:

`return 0;` indica que el programa terminó correctamente.

3. Scripts de build y clean

3.1 build.bat (compilación)

```
@echo off

REM --- Archivos C ---
set FILES=src\glad.c src\main.c

REM --- Librerías ---
set LIBS="E:\NACHO\ESTUDIO\PROYECTOS\PROGRAMACION\GAMEENGINE_GAME\lib\SDL2main.lib" ^
        "E:\NACHO\ESTUDIO\PROYECTOS\PROGRAMACION\GAMEENGINE_GAME\lib\SDL2.lib" ^
        "E:\NACHO\ESTUDIO\PROYECTOS\PROGRAMACION\GAMEENGINE_GAME\lib\freetype.lib"

REM --- Compilación ---
CL /Zi /I "E:\NACHO\ESTUDIO\PROYECTOS\PROGRAMACION\GAMEENGINE_GAME\include" %FILES% /link %LIBS% /OUT:mygame.exe

**Explicación:**
```

• `@echo off` limpia la salida, evitando que se impriman los comandos. • `set FILES=...` define la lista de archivos .c a compilar. • `set LIBS=...` define las librerías estáticas contra las que vamos a enlazar. • La `^` al final de línea permite partir el comando en varias líneas para que sea legible. • El comando `CL` usa el compilador de Microsoft para compilar y enlazar en un solo paso: `- /Zi` genera información de depuración. `- /I "...include"` añade la ruta donde están los headers (SDL2, glad, freetype). `-%FILES%` se expande a `src\glad.c src\main.c`. `-%LIBS%` pasa la lista de librerías al linker. `-%OUT:mygame.exe` indica el nombre del ejecutable final.

3.2 clean.bat (limpieza)

```
@echo off
del *.*.ilk *.*.obj *.*.pdb *.*.exe
```

Este script borra todos los archivos generados por la compilación:

• `*.obj` – archivos objeto intermedios. • `*.pdb` – símbolos de depuración. • `*.ilk` – información de link incremental. • `*.exe` – ejecutables generados (incluido `mygame.exe`).

4. Historial de errores y soluciones

4.1 Problemas al cambiar de unidad y ruta

```
*****
** Visual Studio 2022 Developer Command Prompt v17.14.21
*****
C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools>E:\>E:
Acceso denegado.

...
```

```
C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools>E:
E:\>cd E:\NACHO\ESTUDIO\PROYECTOS\PROGRAMACION\GAMEENGINE_GAME\engine-from-scratch
```

Al principio intentabas hacer `E:\>E:` como si fuera un comando encadenado. En el Developer Command Prompt, hay que hacer dos pasos separados: 1. `E:` para cambiar de unidad. 2. `cd \NACHO\ESTUDIO\...\engine-from-scratch` para ir a la carpeta.

4.2 Error: falta KHR/khrplatform.h

```
glad.c
E:\NACHO\ESTUDIO\PROYECTOS\PROGRAMACION\GAMEENGINE_GAME\include\glad\glad.h(89): fatal error C1083: No se puede abr
main.c
Generando código...
```

Este error indica que glad.h intenta incluir ``, pero el compilador no encontraba esa ruta. La solución fue asegurarse de que el archivo `khrplatform.h` está dentro de la carpeta correcta (`include\glad` en tu caso).

4.3 Compilación correcta con 'Hello there!'

```
E:\...\engine-from-scratch>build && mygame
Compilador de optimización de C/C++ de Microsoft (R) versión 19.44.35221 para x64
```

```
glad.c
main.c
Generando código...
Microsoft (R) Incremental Linker Version 14.44.35221.0
```

```
/out:glad.exe
/debug
E:\...\lib\SDL2main.lib
E:\...\lib\SDL2.lib
E:\...\lib\freetype.lib
/OUT:mygame.exe
glad.obj
main.obj
Hello there!
```

Aquí ya se ve una compilación y link correctos: ningún error y el programa imprime `Hello there!`.

4.4 Error de variables `should\_quit` y `quit`

```
src\main.c(47): error C2146: error de sintaxis: falta ';' delante del identificador 'quit'
src\main.c(47): error C2065: 'quit': identificador no declarado
src\main.c(49): error C2065: 'should_quit': identificador no declarado
src\main.c(56): error C2065: 'should_quit': identificador no declarado
```

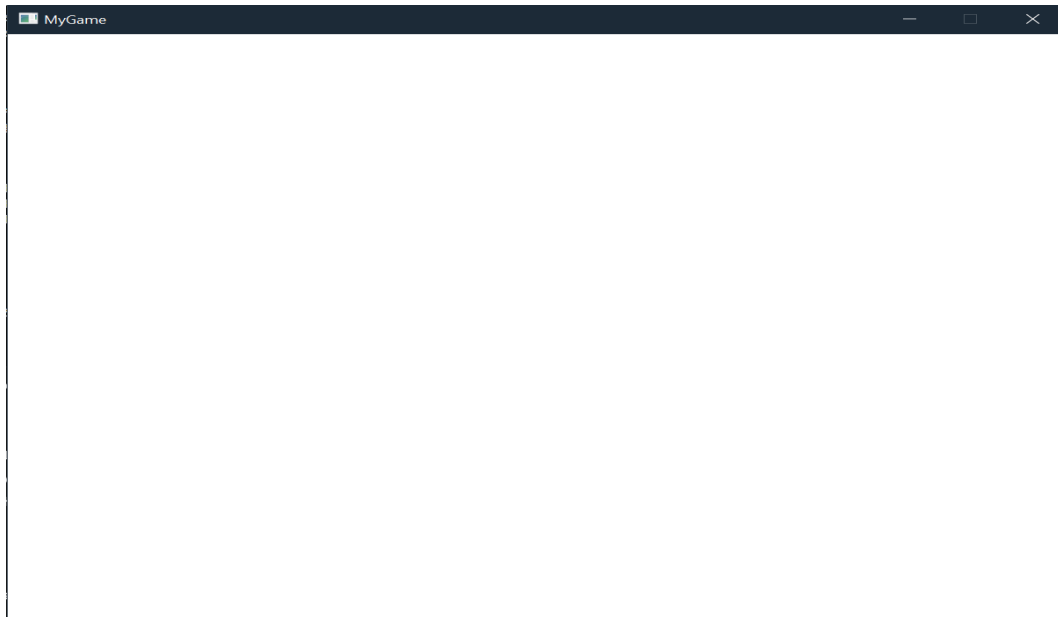
Este error ocurrió cuando empezaste a montar el bucle de juego. Estabas usando las variables `quit` / `should\_quit` sin haberlas declarado antes. La solución fue declarar `bool should\_quit = false;` antes del `while` (!should_quit) y eliminar referencias a una variable `quit` que no existía.

4.5 Ejecución final con OpenGL cargado

```
OpenGL Loaded
Vendor:      NVIDIA Corporation
Renderer:    NVIDIA GeForce RTX 3060 Ti/PCIe/SSE2
Version:     4.6.0 NVIDIA 511.80
Hello there!
```

Esta salida confirma que: • SDL se inicializó bien. • El contexto OpenGL se creó correctamente. • GLAD cargó todas las funciones. • El programa puede consultar información de la GPU y la versión de OpenGL.

5. Ventana resultante



Captura: ventana SDL2 + OpenGL creada por tu código (`MyGame`, 800x600). Es la prueba visual de que toda la configuración (librerías, includes, build.bat) funciona.

6. Conclusión

Has pasado de no tener nada configurado a tener un proyecto C puro que:

- Compila con el toolchain de Visual Studio desde la línea de comandos.
- Usa SDL2 para crear una ventana y manejar eventos.
- Usa GLAD para cargar OpenGL moderno (3.3+).
- Muestra información real de tu GPU y versión de OpenGL.
- Tiene scripts de build y clean propios.

Este documento te sirve como referencia y como base para escribir un README muy sólido en GitHub.